1. Dynamic constant-time audit of the `crypto` crate (2026-11-02)

Study: `research/ct_dynamic_audit_20261102` · **protocol frozen:** `a1530967a` (after the controls dry run, §8) · **run:** task card `CARD-ct-dynamic-audit-20261102-1`, completed (receipt `fd4fee55…`, 2318 s) · **binary:** `ebdf722a…` (release, line tables) · **raw:** `raw.tar.xz` (`01bb9c04…`; extract with `tar -xJf raw.tar.xz`) · **rows:** `results/rows.jsonl` (`2a5c921a…`) · **summary:** `results/summary.json` (`7eb2fa01…`) · **host:** session container, Valgrind 3.22 (instruction-stream metrics, not clocks).

1.1. What was measured

The 2026-10-08 inventory (`docs/audits/SIDE_CHANNEL_AUDIT_20261008.md`) classified secret-handling paths by reading the source and measured nothing. This study executes each path under Valgrind with the secret marked **undefined** (Memcheck: every conditional jump or memory address that depends on it is reported, the ctgrind method) and counts instructions per call (Callgrind) across four secret classes: `aa` (one secret repeated), `random`, `short` ($< 2^{32}$) and `lowhw` (weight 8). Positive control: a secret-dependent trip count plus a secret-indexed load. Negative control: masked fixed-trip arithmetic.

1.2. Controls

The positive control was detected by both instruments (2 taint sites; Ir 69–122). The negative control was clean (no site; Ir constant). Both instruments are valid for this run. The dry run (protocol §8) also established that **a compiler-emitted conditional move is invisible to both instruments**.

1.3. Results: taint (exact witnesses of secret-dependent control flow or addressing)

target	sites	where (first in-crate frame)
x25519	0	none reported: consistent with the fixed-limb `cswap` ladder now on `main`
scalar_mul_secret, P-256	3 (+1 harness)	conversion boundary only: `to_affine` infinity check (p256_point.rs:82), `Uint::to_biguint` (ct_bignum.rs:96), `FieldElement::new` BigUint `%` (field.rs:23); the projective ladder loop itself is not flagged
scalar_mul_secret, secp256k1	3 (+1 harness)	same pattern (secp256k1_point.rs:103, ct_bignum.rs:96, field.rs:23)
scalar_mul_secret, SM2 (generic)	1 (+1 harness)	**point.rs:390: `if bit { … } else { … }` in `Point::scalar_mul_ct`**: the ladder swap branches on every secret bit. The 2026-10-22 fix equalised the work, not the control flow
mod_inverse_prime_ct (ECDSA k^{-1})	5 (+1 harness)	zero test (utils/mod.rs:94) and BigUint arithmetic inside `mod_pow_ct` (mod.rs:63, 68–70)
mod_inverse (SM2 $(1+d)^{-1}$)	7 (+1 harness)	extended Euclid `mod_inverse_bigint` (mod.rs:132–161), as the 2026-10-08 audit ranked
ed25519_sign	7	**double-and-add on the secret scalar** (ed25519.rs:190, 192) plus scalar encoding and signing arithmetic (ed25519.rs:295, 311, 364, 368)
ecdsa::sign (P-256)	19	scalar arithmetic in `sign_hash` (ecdsa.rs:52–86), `bits2int`, blinding (`ct.rs:83`), and the conversion sites above
sm2::sign	11	`sign` (sm2.rs:150–158), `mod_inverse` (mod.rs:132–161)

"+1 harness" is `main.rs:145`, the harness's own `BigUint::from_bytes_be(secret)`: any caller of these APIs, which take secrets as `BigUint`, executes that normalisation. It is an API-boundary site, not a library line.

1.4. Results: instruction count per call

Under the frozen rule (§6) the Ir verdict is **"no Ir variation observed across classes"** for x25519 and both controls (x25519: 911 792 instructions on every one of 80 inputs), and **"not applicable: A/A not deterministic"** for every other target. The A/A arm repeats one secret and still varies by 6 to 208 771 instructions. This is expected for the targets that draw OsRng (ECDSA blinding, the SM2 nonce). For the deterministic paths it most likely reflects heap-allocator state inside num-bigint: a per-call Ir comparison cannot separate that from the secret, so the rule declines to.

Exploratory, outside the decision rule (read against each target's own A/A spread): `mod_inverse` runs 17 900–28 376 instructions on `short` secrets against 326 399–430 424 on `random` (A/A spread 6); `ed25519_sign` spans 17.5 M instructions across `random` secrets (A/A spread 47 k); `mod_inverse_prime_ct` spans 15 k (A/A spread 71). The dedicated P-256 and secp256k1 paths run about 400 fewer instructions on `short` scalars, with class ranges disjoint from `random` (A/A spread 80). That is a small bit-length effect most likely at the BigUint conversion boundary. The generic SM2 ladder shows no class effect beyond its A/A spread, consistent with the 2026-10-22 length fix.



Figure 1: Per-call instruction count, deviation from each target's minimum (square-root scale), by secret class; right column: taint sites. Recorded values from `results/summary.json`, card `-1`.

1.5. Status of the 2026-10-08 findings (dated correction, 2026-11-02)

The 2026-10-08 inventory said, from the source, that X25519's "ladder swap is a branch". On `main` (`46f01e51d`) X25519 uses fixed limbs and `cswap`, and this run reports no secret-dependent site for it. The other ranked findings are **confirmed dynamically**: signature scalar arithmetic in num-bigint (ECDSA, SM2), extended-Euclid inversion of the private key (SM2), Ed25519 double-and-add, and the generic ladder used by SM2/GOST/EC-KCDSA/EC-ElGamal. That ladder still branches on the secret bit after the 2026-10-22 work-count fix.

1.6. Execution record

The card completed (4/4 commands, exit 0; all digests on disk equal the receipt; 11 Memcheck files and 80 per-call Callgrind dumps per target; no Valgrind errors in `raw/stderr.log`). The director's write-once assessment (`cards/ASSESSMENT-20261102.md`) records `relay_verbatim: false` because `task_card.py check-relay` reported two output lines missing. Its own diagnosis: `run` prints a command's outputs in the card's declared order, while `check-relay` rebuilds them from the receipt JSON written with sorted keys; recomputed in card order, nothing is missing and every digest matches. **The main session treats the relay as verbatim for this reason** (a tool ordering artifact affecting any card with two or more declared outputs, reported upstream as a `task_card.py` defect), and leaves the assessment file unchanged.

1.7. Scope and limits

Taint covers this build (release, line tables), these inputs (4 calls per target) and Memcheck's model. It does not see conditional moves, cache-line or port effects, or anything the compiler emits only as `cmov`. No target is claimed "constant-time" or "timing-safe". No wall-clock study was run: the EKS route (AGENTS.md §9a) was unavailable (no AWS identity), and a dudect-style clock study on an isolated host is the follow-up. No independent `review-run` record exists yet.

1.8. Next technical steps

1. Replace the `if bit` swap in `Point::scalar_mul_ct` (point.rs:390) with a masked conditional swap, and re-run this audit as the regression test.
2. Route Ed25519 signing through a fixed-window or ladder scalar multiplication. Move ECDSA/SM2 scalar arithmetic mod n onto `ct_bignum` with a Fermat inverse.
3. Add a Callgrind mode that counts inside the library call only, excluding allocator paths, so the Ir test becomes applicable to the deterministic BigUint targets.